

ソフトウェア工学 レポート 1

情報 開発

2022311042

愛知県立大学 情報科学部 情報科学科

神野友哉

2024 年 7 月 7 日

1 はじめに

Java 言語において、コンポジットパターン未適用の場合でジェネリクスを用いたコードを作成した。また、そのデバッグ用の出力と、 Powershell 等端末で実行するためのコマンドも併記する。

2 実験結果

クラスは、パターン未適用のコードで City、Prefecture、Region、Country、SoftwareNotPatern。main 関数があり、その中で自治体の名称や人工を設定するのが、SoftwareNotPatern である。コンパイル・実行は図 1 のように行えばよい。その際に、ソースコードと同じ階層である必要がある。

```
PS C:\Users\tomotomo\Desktop\プログラム\latex\software\java> javac SoftwareNotPatern.java
PS C:\Users\tomotomo\Desktop\プログラム\latex\software\java> java SoftwareNotPatern
名古屋市 : 2328846人, 豊田市 : 425677人, 岡崎市 : 387835人, 愛知県 : 3142358人,
浜松市 : 797980人, 静岡市 : 704898人, 富士市 : 248399人, 静岡県 : 1751277人,
中部地方 : 4893635人,
大阪市 : 2756034人, 堺市 : 822671人, 東大阪市 : 489151人, 大阪府 : 4067856人,
京都市 : 1388807人, 宇治氏 : 183510人, 亀岡市 : 87518人, 京都府 : 1659835人,
近畿地方 : 5727691人,
日本国 : 10621326人,
```

図 1: 実行したコマンドと動作確認用のデバッグ出力

2.1 作成したプログラム

source 1: パターン未適用 City.java

```
public class City {
    private String name;
    private int population;

    public City(String name, int population) {
        this.name = name;
        this.population = population;
    }
    public String getName() {return this.name;}
    public void setName(String name) {this.name = name;}
    public int getPopulation() {
```

```

        System.out.print(this.name + " : " + this.population + "人, ");
        return this.population;
    }
    public void setPopulation(int population) {this.population = population;}
}

```

source 2: パターン未適用 Prefecture.java

```

import java.util.*;

public class Prefecture {
    List<City> cities = new ArrayList<>();
    private String name;

    public Prefecture(String name) {this.name = name;}
    public String getName() {return this.name;}
    public void setName(String name) {this.name = name;}
    public void add(City city) {cities.add(city);}
    public int getPopulation() {
        int result = 0;
        for (int i = 0; i < cities.size(); i++) {
            City city = (City) cities.get(i);
            result += city.getPopulation();
        }
        System.out.print(this.getName() + " : " + result + "人, ");
        System.out.println();
        return result;
    }
}

```

source 3: パターン未適用 Region.java

```

import java.util.*;

public class Region {
    List<Prefecture> prefectures = new ArrayList<>();
    private String name;

    public Region(String name) {this.name = name;}
    public String getName() {return this.name;}
    public void setName(String name) {this.name = name;}
    public void add(Prefecture prefecture) {prefectures.add(prefecture);}
    public int getPopulation() {
        int result = 0;
        for (int i = 0; i < prefectures.size(); i++) {
            Prefecture prefecture = (Prefecture) prefectures.get(i);
            result += prefecture.getPopulation();
        }
        System.out.print(this.getName() + " : " + result + "人, ");
        System.out.println("\n");
        return result;
    }
}

```

source 4: パターン未適用 Country.java

```

import java.util.*;

public class Country {
    List<Region> regions = new ArrayList<>();
    private String name;

    public Country(String name) {this.name = name;}
    public String getName() {return this.name;}
    public void setName(String name) {this.name = name;}
    public void add(Region region) {regions.add(region);}
    public int getPopulation() {
        int result = 0;
        for (int i = 0; i < regions.size(); i++) {
            Region region = (Region) regions.get(i);
            result += region.getPopulation();
        }
        System.out.print(this.getName() + " : " + result + "人, ");
        System.out.println("\n");
    }
}

```

```

    }
    return result;
}

```

source 5: パターン未適用 SoftwareNotPatern.java

```

import java.util.*;

public class SoftwareNotPatern {

    public static void main(String[] args) {

        Country country = new Country("日本国");
        Region region1 = new Region("中部地方");
        Region region2 = new Region("近畿地方");
        Prefecture prefecture1 = new Prefecture("愛知県");
        Prefecture prefecture2 = new Prefecture("静岡県");
        Prefecture prefecture3 = new Prefecture("大阪府");
        Prefecture prefecture4 = new Prefecture("京都府");

        prefecture1.add(new City("名古屋市", 2328846));
        prefecture1.add(new City("豊田市", 425677));
        prefecture1.add(new City("岡崎市", 387835));
        region1.add(prefecture1);
        prefecture2.add(new City("浜松市", 797980));
        prefecture2.add(new City("静岡市", 704898));
        prefecture2.add(new City("富士市", 248399));
        region1.add(prefecture2);

        prefecture3.add(new City("大阪市", 2756034));
        prefecture3.add(new City("堺市", 822671));
        prefecture3.add(new City("東大阪市", 489151));
        region2.add(prefecture3);
        prefecture4.add(new City("京都市", 1388807));
        prefecture4.add(new City("宇治市", 183510));
        prefecture4.add(new City("亀岡市", 87518));
        region2.add(prefecture4);

        country.add(region1);
        country.add(region2);
        int x = country.getPopulation();
    }
}

```

3 考察

上記コードの `List<●●> □□ = new ArrayList<>();` となっている部分において、`<>`の部分がふたつ存在する。この両方を消したものが Generics 未使用のコードとなる。その場合に、`javac -Xlink:unchecked SoftwareNotPatern.java` というコマンドを実行してコンパイルしていた。しかし、Generics を使用すると、コンパイルコマンドが `javac SoftwareNotPatern.java` という形式に短縮されている。これは、型の警告を無視するようなオプションであると考えられる。また、動作としてはそこまで変化しているように感じられなかった。リーフから根に至るように横探索で数字を加算し、高さを上げていくのである。例えば、中間ノードとして愛知県や静岡県があれば、それらの人口を計算するためにはそれぞれの県に属する市の人口を全て加算して、やっとのことでそれらの県の人口が判明する。そのため、下位のノードのすべての人口の和が計算できるまで、その中間ノードの上に上がることはできない。

4 感想

変更せよとのことだったので、変更したコードを貼りました。